

13주 차 보고서

(2026년 3월 22일 ~ 2026년 3월 28일)

작성자

박신우

이번 주 한 일

03.23 트럭과 플레이어 같이 날아가는 문제 해결

지난주에 트럭이 트렁크에 탑승한채로 여기저기 비비면 트럭과 함께 날아가는 문제가 생겼다.

그것을 해결하기 위해 충돌 박스를 트렁크에 따로 만들기로 결정을 하였다.

트럭의 생성자에서 아래와 같이 박스 컴포넌트를 설정해 주었고.

람다함수로 각 박스들이 다음과 같은 기능을 하도록 하였다.

SetCollisionEnabled(QueryAndPhysics)

→ 충돌 판정이 가능하도록 활성화

SetCollisionResponseToAllChannels(ECR_Ignore)

→ 기본적으로 다른 모든 오브젝트와의 충돌은 무시

SetCollisionResponseToChannel(ECC_Pawn, ECR_Block)

→ 캐릭터(Pawn)만 막도록 설정

SetGenerateOverlapEvents(false)

→ 겹침 이벤트는 발생시키지 않음

```
CargoFloorCollision = CreateDefaultSubobject<UBoxComponent>(TEXT("CargoFloorCollision"));
CargoFloorCollision->SetupAttachment(RootComponent);

CargoLeftWallCollision = CreateDefaultSubobject<UBoxComponent>(TEXT("CargoLeftWallCollision"));
CargoLeftWallCollision->SetupAttachment(RootComponent);

CargoRightWallCollision = CreateDefaultSubobject<UBoxComponent>(TEXT("CargoRightWallCollision"));
CargoRightWallCollision->SetupAttachment(RootComponent);

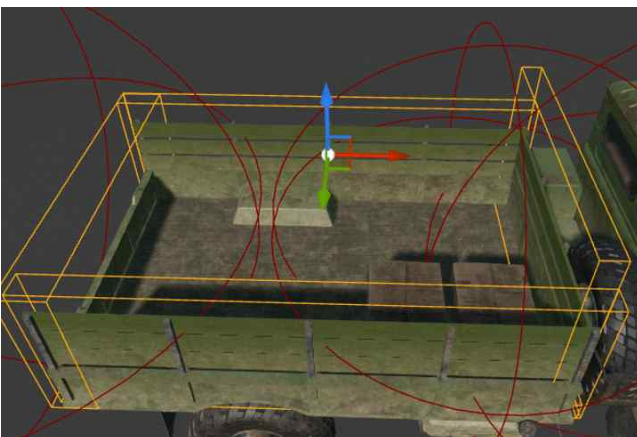
CargoFrontWallCollision = CreateDefaultSubobject<UBoxComponent>(TEXT("CargoFrontWallCollision"));
CargoFrontWallCollision->SetupAttachment(RootComponent);

CargoBackWallCollision = CreateDefaultSubobject<UBoxComponent>(TEXT("CargoBackWallCollision"));
CargoBackWallCollision->SetupAttachment(RootComponent);

auto SetupCargoCollision = [](UBoxComponent* Box)
{
    Box->SetCollisionEnabled(ECollisionEnabled::QueryAndPhysics);
    Box->SetCollisionResponseToAllChannels(ECR_Ignore);
    Box->SetCollisionResponseToChannel(ECC_Pawn, ECR_Block);
    Box->SetGenerateOverlapEvents(false);
};

SetupCargoCollision(CargoFloorCollision);
SetupCargoCollision(CargoLeftWallCollision);
SetupCargoCollision(CargoRightWallCollision);
SetupCargoCollision(CargoFrontWallCollision);
SetupCargoCollision(CargoBackWallCollision);
```

에디터에서 다음과 같이 크기를 조절해 주었다.



이번 주 한 일

이제 트렁크에서 특정 범위 밖으로 캐릭터가 나가지 못하게 만들어,
캐릭터랑 트럭이 비벼지며 날아가는 현상이 발생하지 않아졌다.

03.24 트럭 장착 기관총 클래스 생성

트럭에 장착할 기관총 클래스를 생성하였다.

우선, 생성자에서 좌우 회전축, 상하 회전축, 기관총 메쉬, 총알 발사 포인트를 생성하게
하였고,

```
AMountedMachineGun::AMountedMachineGun()
{
    PrimaryActorTick.bCanEverTick = true;

    SceneRoot = CreateDefaultSubobject<USceneComponent>(TEXT("SceneRoot"));
    SetRootComponent(SceneRoot);

    YawPivot = CreateDefaultSubobject<USceneComponent>(TEXT("YawPivot"));
    YawPivot->SetupAttachment(SceneRoot);

    PitchPivot = CreateDefaultSubobject<USceneComponent>(TEXT("PitchPivot"));
    PitchPivot->SetupAttachment(YawPivot);

    GunMesh = CreateDefaultSubobject<USkeletalMeshComponent>(TEXT("GunMesh"));
    GunMesh->SetupAttachment(PitchPivot);
    GunMesh->SetCollisionEnabled(ECollisionEnabled::NoCollision);

    MuzzlePoint = CreateDefaultSubobject<USceneComponent>(TEXT("MuzzlePoint"));
    MuzzlePoint->SetupAttachment(PitchPivot);
    MuzzlePoint->SetRelativeLocation(FVector(150.0f, 0.0f, 0.0f));
}
```

유저 setter를 통해 기관총 사용하는 유저가 누군지 업데이트 시켜주었다.

```
void AMountedMachineGun::SetWeaponUser(AFPSBaseCharacter* NewUser)
{
    CurrentUser = NewUser;
}
```

총알 발사하는 fire 함수에서는 총기 발사 텀을 정해주었다.

```
// 무작정 연사가 안되도록 연사 텀 설정
const float CurrentTime = GetWorld()->GetTimeSeconds();
if (CurrentTime - LastFireTime < FireInterval)
{
    return;
}
LastFireTime = CurrentTime;
```

이번 주 한 일

총알이 발사되는 방향을 결정하기 위해,
플레이어의 눈 위치와 방향을 가져와주었고,

```
FVector CameraLocation;  
FRotator CameraRotation;  
CurrentUser->GetActorEyesViewPoint(CameraLocation, CameraRotation);
```

그 다음,

플레이어가 바라보는 방향으로 10000까지 거리의 선을 쏜다.

그 라인트레이스로 맞은 지점을 찾는다.

무언가 맞았다면? -> 그 맞은 지점이 목표

아무것도 안맞았다면 -> 먼 거리 끝 점이 목표

```
const FVector TraceStart = CameraLocation;  
const FVector TraceEnd = TraceStart + CameraRotation.Vector() * 10000.0f;  
  
FHitResult HitResult;  
FCollisionQueryParams QueryParams;  
QueryParams.AddIgnoredActor(this);  
QueryParams.AddIgnoredActor(CurrentUser);  
  
// 라인트레이스 하여 맞으면 hit  
const bool bHit = GetWorld()->LineTraceSingleByChannel(HitResult, TraceStart, TraceEnd, ECC_Visibility, QueryParams);  
// 아무것도 안맞았다면, 멀리 쏘버리기  
const FVector TargetLocation = bHit ? HitResult.Location : TraceEnd;
```

총기의 총알 발사 위치는 생성자에서 만들고, 블루프린트에서 조정 가능한 MuzzlePoint다.
총구 위치와, 목표지점을 이용해 플레이어가 바라보는 방향으로 총알이 날아가게 하는
방향 벡터를 구했다.

```
const FVector FireLocation = MuzzlePoint ? MuzzlePoint->GetComponentLocation() : GetActorLocation();  
const FVector FireDirection = (TargetLocation - FireLocation).GetSafeNormal();  
const FRotator FireRotation = FireDirection.Rotation();
```

기존에 만들어둔 풀링 시스템이 되어있으면 풀에서 꺼내서 총알을 쓰고,
아니면 그냥 총알을 생성하도록 하였다.

```
if (UObjectPoolSubsystem* PoolSubsystem = GetWorld()->GetSubsystem<UObjectPoolSubsystem>())  
{  
    if (AActor* PooledActor = PoolSubsystem->SpawnFromPool(ProjectileClass, FireLocation, FireRotation))  
    {  
        if (AFPSProjectile* Projectile = Cast<AFPSProjectile>(PooledActor))  
        {  
            Projectile->SetOwner(this);  
            Projectile->SetInstigator(CurrentUser);  
            Projectile->FireInDirection(FireDirection);  
        }  
    }  
}  
else  
{  
    if (AFPSProjectile* Projectile = GetWorld()->SpawnActor<AFPSProjectile>(ProjectileClass, FireLocation, FireRotation))  
    {  
        Projectile->SetOwner(this);  
        Projectile->SetInstigator(CurrentUser);  
        Projectile->FireInDirection(FireDirection);  
    }  
}
```


이번 주 한 일

기관총이 총알을 잘 쏘는 것을 확인할 수 있었다.

03.25 트럭 장착 기관총 문제 해결

여러 가지 문제가 발생하였다.

1. 기관총 총알이 가끔 트럭을 쏘버림.
2. 총알이 약간 이상한 방향으로 발사됨
3. 기관총을 조종할 때 카메라를 회전해도 기관총이 회전하지 않음.

우선 1번 문제를 해결은 간단하게 하였다.

기관총의 fire 함수에서 라인트레이스 할 때 트럭을 무시하도록 하였고,

```
queryParams.AddIgnoredActor(this);  
queryParams.AddIgnoredActor(CurrentUser);  
if (CurrentUser->CurrentTruck)  
{  
    queryParams.AddIgnoredActor(CurrentUser->CurrentTruck);  
}
```

발사체를 생성한 후에도 충돌처리 안하도록 해주었다.

```
Projectile->CollisionComponent->IgnoreActorWhenMoving(this, true);  
Projectile->CollisionComponent->IgnoreActorWhenMoving(CurrentUser, true);  
if (CurrentUser->CurrentTruck)  
{  
    Projectile->CollisionComponent->IgnoreActorWhenMoving(CurrentUser->CurrentTruck, true);  
}
```

2번 문제는 muzzle point를 내가 좀 위쪽에 설정해줬었다.....

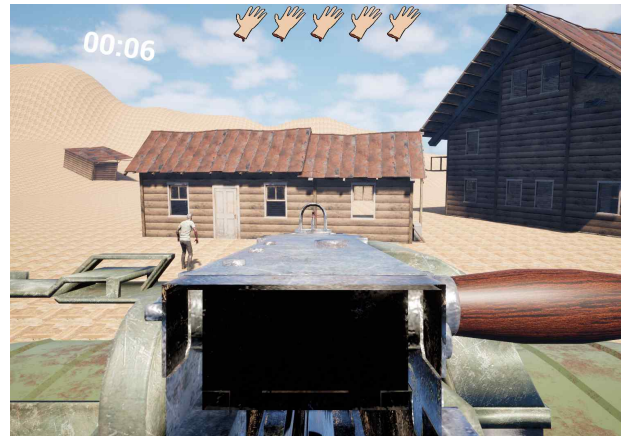
그리고 추가적으로 윤진이가 만든 UI를 기관총에 장착해주었다.



이번 주 한 일

03.26~27 기관총 회전

아래 사진과 같이 정면을 볼 때는 괜찮아보이는데, 회전을 하면 좀 이상한 문제가 있다.



우선, 기존에는 카메라가 기관총 조준선을 따라간 것이 아니라,

기관총의 회전만 따라보는 식이었다.

기관총 생성자에서 기관총의 PitchPivot에 조준용 스프링암과 카메라 포인트를 붙였다.

```
CameraBoom = CreateDefaultSubobject<USpringArmComponent>(TEXT("CameraBoom"));
CameraBoom->SetupAttachment(PitchPivot);
CameraBoom->TargetArmLength = 0.0f;
CameraBoom->bDoCollisionTest = false;
CameraBoom->bUsePawnControlRotation = false;
CameraBoom->bEnableCameraLag = true;
CameraBoom->bEnableCameraRotationLag = true;
CameraBoom->CameraLagSpeed = CameraLocationLagSpeed;
CameraBoom->CameraRotationLagSpeed = CameraRotationLagSpeed;
CameraBoom->SocketOffset = CameraSocketOffset;

CameraPoint = CreateDefaultSubobject<USceneComponent>(TEXT("CameraPoint"));
CameraPoint->SetupAttachment(CameraBoom, USpringArmComponent::SocketName);
```

기관총 회전을 업데이트하는 , UpdateAim() 함수를 만들어주었다.

우선, 기관총의 좌우 회전 목표값을 계산하였다.

이어서 상하 회전 목표값도 계산을 하였다.

아래 코드는 플레이어가 보고 싶은 방향으로 얼마나 돌아야하는 지 계산하고, 최대, 최소 각도 안으로 제한하는 코드이다.

```
// 기관총이 어디를 봐야 하는지 계산
const FRotator ActorRotation = GetActorRotation();
// -120 , 120도 까지만 회전 하도록
// FindDeltaAngleDegrees -> 플레이어 방향과 기관총 방향이 얼마나 차이 나는지 계산
TargetRelativeYaw = FMath::Clamp(FMath::FindDeltaAngleDegrees(ActorRotation.Yaw, ControlRotation.Yaw), MinYaw, MaxYaw);
TargetRelativePitch = FMath::ClampAngle(ControlRotation.Pitch, MinPitch, MaxPitch);
```

이번 주 한 일

플레이어 방향으로 기관총이 바로 바로 회전하면 끊어지는 느낌이 들음으로, 보간을 하여 회전하도록 해주었다.

현재값 : CurrentRelative

목표값 : TargetRelative

를 FInterpTo(Current, Target, DeltaSeconds, Speed)

현재값을 목표값으로 부드럽게 이동해주는 보간 함수에 넣어주었다.

```
// 부드럽게 회전하도록 보간을 해줌
const float DeltaSeconds = GetWorld() ? GetWorld()->GetDeltaSeconds() : 0.0f;
CurrentRelativeYaw = FMath::FInterpTo(CurrentRelativeYaw, TargetRelativeYaw, DeltaSeconds, YawInterpSpeed);
CurrentRelativePitch = FMath::FInterpTo(CurrentRelativePitch, TargetRelativePitch, DeltaSeconds, PitchInterpSpeed);
```

이제 회전값을 넣어 주었다.

Yaw 는 좌우만, Pitch는 상하만 회전에 관여하도록 하여, 더욱 거치형 기관총 느낌이 나도록 만들어 주었다.

```
if (YawPivot)
{
    YawPivot->SetRelativeRotation(FRotator(0.0f, CurrentRelativeYaw, 0.0f));
}

if (PitchPivot)
{
    PitchPivot->SetRelativeRotation(FRotator(CurrentRelativePitch, 0.0f, 0.0f));
}
```

카메라 붐도 업데이트를 시켜주어, 기관총을 잘 따라다니도록 만들어 주었다.

```
if (CameraBoom)
{
    CameraBoom->CameraLagSpeed = CameraLocationLagSpeed;
    CameraBoom->CameraRotationLagSpeed = CameraRotationLagSpeed;
    CameraBoom->SocketOffset = CameraSocketOffset;
}
```

03.28 보고서 정리

다음 주 할 일

- 트럭 탑승, 하차 로직 재검토
- 트럭 운전자가 좌석에 앉아서 운전하도록
- 총알이 조금 더 정확히 가도록.
- 소총 쏘인

특이사항

이번주에 좀 열심히 한 거 같아서 뿌듯했다

13주 차 보고서

(2026년 3 월 22 일 ~ 2026년 3 월 28 일)

작성자

배주환

이번 주 회의 내용

이번 주 한 일

전에 강의를 공부하면서 임의로 만들었던 프로토콜을 우리의 게임에 맞게 다시 제대로 만들고, 하나씩 추가할 예정이다.


```

syntax = "proto3";
package Protocol;

enum ObjectType
{
    OBJECT_TYPE_NONE = 0;
    OBJECT_TYPE_CREATURE = 1;      // 유저, 좀비
    OBJECT_TYPE_PROJECTILE = 2;    // 총알, 날라가는 투사체
    OBJECT_TYPE_ITEM = 3;         // 바닥에 있는 아이템
}

enum CreatureType
{
    CREATURE_TYPE_NONE = 0;
    CREATURE_TYPE_PLAYER = 1;
    CREATURE_TYPE_ZOMBIE = 2;
}

enum PlayerType
{
    PLAYER_TYPE_NONE = 0;
    PLAYER_TYPE_SHOOTER = 1;
}

enum ZombieType
{
    ZOMBIE_TYPE_NONE = 0;
    ZOMBIE_TYPE_MELEE = 1;
    ZOMBIE_TYPE_RANGED = 2;
    ZOMBIE_TYPE_TANKER = 3;
}

enum MoveState
{
    MOVE_STATE_NONE = 0;
    MOVE_STATE_IDLE = 1;
    MOVE_STATE_RUN = 2;
    MOVE_STATE_JUMP = 3;
    MOVE_STATE_FALL = 4;      // 점프 후 떨어질때
    MOVE_STATE_DEAD = 6;
}

enum WeaponType
{
    WEAPON_TYPE_NONE = 0;
    WEAPON_TYPE_RIFLE = 1;
}

```

생물체는 플레이어와 좀비밖에 없어서 변동이 없을 예정이고, 아이템은 여러가지가 있는데 아직은 총밖에 없어서 추후에 추가할 예정이다.

```

message C_EQUIP_WEAPON
{
    uint64 itemObjectId = 1;
}

message S_EQUIP_WEAPON
{
    uint64 playerId = 1;      // 누가 주웠는지 (이 사람 손에 총을 붙여야 함)
    uint64 itemObjectId = 2;  // 맵에 떨어져 있던 어떤 아이템을 주웠는지 (주웠으니 맵에서 지워야 함)
    int32 weaponType = 3;    // 그 아이템이 무슨 총인지 (어떤 모델링을 띄울지 결정)
}

```

총을 주웠을때에 대한 프로토콜도 만들어주었다.

```

message PosInfo
{
    uint64 object_id = 1;
    float x = 2;
    float y = 3;
    float z = 4;
    float yaw = 5;
    MoveState state = 6;
}

message ObjectInfo
{
    uint64 object_id = 1;    // 고유 id
    ObjectType object_type = 2; // 유저인지 좀비인지 아이템인지
    PosInfo pos_info = 3;    // 현재 위치 및 상태
    int32 weapon_type = 4;
}

```

아직은 이동과 총만 있다.



이번주에는 총을 주웠을 때 상대방 화면에서 총을 주운 모습이 제대로 보이도록 했다. 처음에는 총이 이상한 위치에 착용이 되어서 혼자서 각도를 조정해보았는데



여전히 이상한 위치에 있어서 클라 친구에게 어떻게 위치를 조절했는지 묻고 바로 해결했다.

```
void AFPSBaseCharacter::EquipWeaponFromField(AWeaponBase* Weapon)
{
    if (Weapon == nullptr) return;

    // 1. 바닥에 고정되어 있던 물리나 충돌을 끄기
    Weapon->SetActorEnableCollision(false);

    // 2. [소켓에 부착 (SnapToTarget을 써야 소켓 위치로 순간이동합니다.)
    const FName SocketName = TEXT("Gun_socket");
    Weapon->AttachToComponent(Parent::GetMesh(), FAttachmentTransformRules::SnapToTargetNotIncludingScale, SocketName);
    // 3. 위치 조정 (Location)
    Weapon->SetActorRelativeLocation(FVector(InX:-35.209697f, InY:2.353551f, InZ:0.508678f));

    // 회전 조정 (Rotation - Pitch, Yaw, Roll 순서)
    Weapon->SetActorRelativeRotation(FRotator(InPitch:1.090108f, InYaw:-88.966904f, InRoll:-4.015320f));

    // 스케일 조정 (Scale) - 총이 0.15배로 작아져야 하니까!
    Weapon->SetActorRelativeScale3D(FVector(InX:0.15f, InY:0.15f, InZ:0.15f));

    // 4. 캐릭터 변수 업데이트 및 애니메이션Instance 변경
    SetCurrentWeapon(Weapon);

    UE_LOG(LogTemp, Log, TEXT("[Network] %s가 바닥에 있던 무기(%s)를 장착했습니다."), *GetName(), *Weapon->GetName());
}
```




역시 모르는게 있으면 바로바로 물어봐야한다.

```
bool Handle_C_EQUIP_WEAPON(PacketSessionRef& session, Protocol::C_EQUIP_WEAPON& pkt)
{
    GameSessionRef gameSession = static_pointer_cast<GameSession>(session);
    PlayerRef player = gameSession->player.load();
    if (player == nullptr)
        return false;

    // 플레이어가 속한 방(Room)을 찾을
    RoomRef room = player->room.load().lock();
    if (room == nullptr)
        return false;

    // 방에 넣길 때 임시 객체로 복사
    room->DoAsync(&Room::HandleEquipWeapon, player, Protocol::C_EQUIP_WEAPON(pkt));

    return true;
}
```

```
bool Handle_S_EQUIP_WEAPON(PacketSessionRef& session, Protocol::S_EQUIP_WEAPON& pkt)
{
    if (auto* GameInstance = Cast<UFPSProjectGameInstance>(Src: GWorld->GetGameInstance()))
    {
        //일단 서버를 돌아서 내 클라까지 패킷이 잘 도착했는지 로그로 확인!
        //UE_LOG(LogTemp, Warning, TEXT("===== [네트워크] S_EQUIP_WEAPON 수신 ====="));
        //UE_LOG(LogTemp, Warning, TEXT("누가 주웠는가(PlayerID) : %llu"), pkt.playerid());
        //UE_LOG(LogTemp, Warning, TEXT("무슨 아이템(ItemID) : %llu"), pkt.itemobjectid());
        //UE_LOG(LogTemp, Warning, TEXT("무기 타입(WeaponType) : %d"), pkt.weaponType());

        // GameInstance에 함수를 만들어서 실제 모델링을 손에 붙이기
        GameInstance->HandleEquipWeapon(pkt);
    }

    return true;
}
```


패킷 송수신을 확인한 후에

```
void UFPSProjectGameInstance::HandleEquipWeapon(const Protocol::S_EQUIP_WEAPON& pkt)
{
    uint64 PlayerId = pkt.playerid();
    uint64 ItemId = pkt.itemobjectid();

    // 누가 주었는지 찾기
    AFPSBaseCharacter* TargetPlayer = Players.Contains(PlayerId) ? Players[PlayerId] : nullptr;

    // 바닥에 있는 총 찾기 (FieldItems 맵에 등록해둔 것)
    if (FieldItems.Contains(ItemId))
    {
        AWeaponBase* WeaponActor = Cast<AWeaponBase>(FieldItems[ItemId]);

        if (TargetPlayer && WeaponActor)
        {
            // 내 캐릭터라면 이미 로컬에서 처리가 되었겠지만,
            // 혹시 모를 동기화를 위해 남의 캐릭터일 때만 실행
            if (!TargetPlayer->IsLocallyControlled())
            {
                TargetPlayer->EquipWeaponFromField(WeaponActor);
            }

            // 이제 바닥에 없으니 관리 목록에서 제거!
            FieldItems.Remove(ItemId);
        }
    }
}
```

```
void Room::Broadcast(SendBufferRef sendBuffer, uint64 exceptId)
{
    for (auto& item : pair<...> : _objects)
    {
        PlayerRef player = dynamic_pointer_cast<Player>(item.second);
        if (player == nullptr)
            continue;
        if (player->objectInfo->object_id() == exceptId)
            continue;

        if (GameSessionRef session = player->session.lock())
            session->Send(sendBuffer);
    }
}
```

상대방 화면에서도 총이 정확한 위치와 각도로 부착되어서 렌더링 되도록 동기화 처리를 했다.

다음 주 할 일

플레이어가 차량 앞자리에 탑승했을 때와 트렁크에 탑승했을 때의 위치를 보내줄 예정이다.

13주 차 보고서

(2026년 3 월 22 일 ~ 2026년 3 월 28 일)

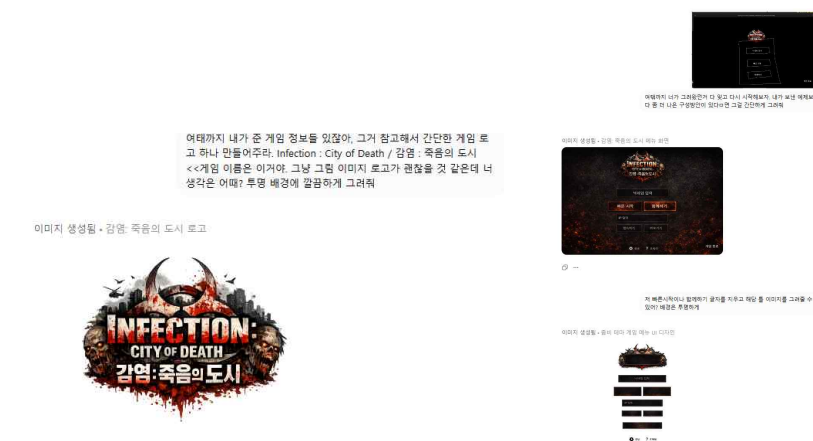
작성자

안윤진

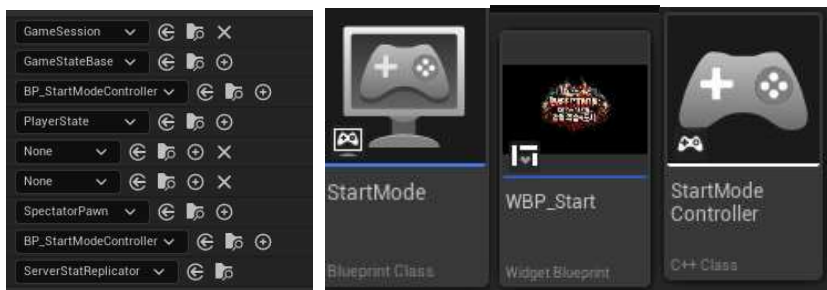
이번 주 한 일



게임 이미지를 만들어 보겠다고, 첫 번째 그림을 레퍼런스를 주고 ai들에게 오른쪽 이미지들을 얻었다. gpt 티 난다고 팀원들에게 막 좋은 평은 받지 못했다.

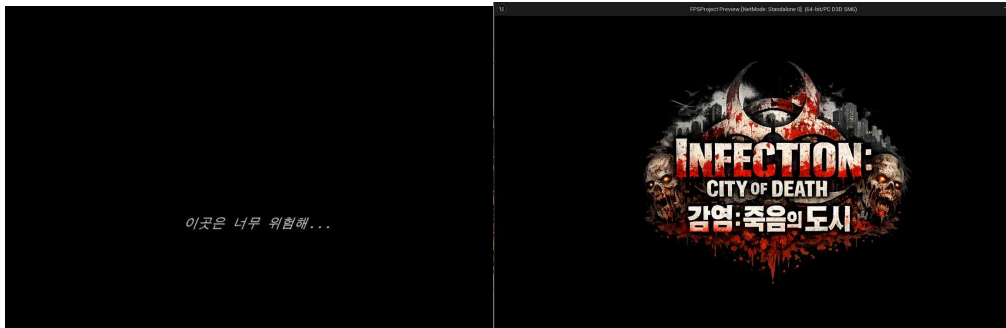


gpt에게 게임 이름을 줘서 로고도 만들고, 레퍼런스를 주며 시작창 세팅에 사용할 패널 이미지도 얻었다.

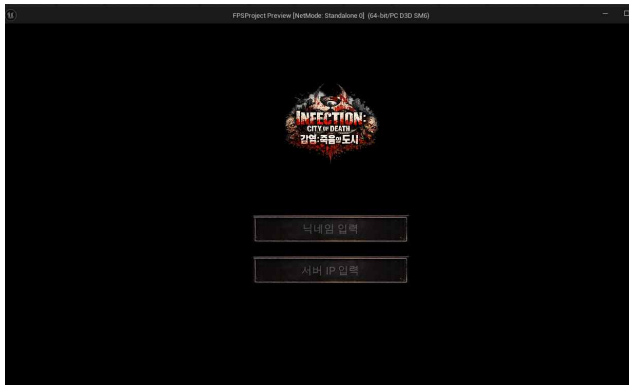


- 게임 플레이와 시작(닉네임 입력받고 하는 부분)은 모드 자체를 아예 분리하였다.
모드를 추가로 만들고, 각 모드별 컨트롤러 역할이 아예 다르기에 플레이어 컨트롤러도 새로 만들었다.
처음엔 원래 쓰던 것에 ui만 연결했었다. 하지만 그렇게 해보니 플레이어는 계속 움직이고 다른 ui도 함께 뜨는 등, 제외할 것이 너무 많아져 역할 분리를 하였다. 새로 만든 플레이어에 ui를 달아 ip주소와 닉네임 입력 받는 것까지 진행할 예정이다.

이번 주 한 일



웃기지만 오프닝 화면(?)도 만들었다. 약간의 로고 애니메이션이 들어가있다.



닉네임과 게임 서버 ip 입력받는 창도 제작했다.

이 이후로 진짜 지옥이었다. 머지에서 문제가 정말 많이 생겼다.

화요일 저녁에 머지 시도 -> 머지 후 몇몇 파일들이 없다고 떠 다음날 만나서 해결하기로 함.

수요일 -> 서버에서 여러 파일들이 깃허브를 통해 오지 않음. 복붙 등을 통해 없는 파일들을 다시 만들고 다시 머지, -> 새로 생긴 파일들에 대한 찾을 수 없다.. 문제 발생 -> 경로 수정 및 지피티와 여러 시도

-> 일반적인 경로 문제 아닌 것 같음. 지피티와 여러 시도로 후 다른 방도로 언리얼 삭제 및 다시 설치.. 이것 외에도 여러 시도를 통해 5시간 정도 소요. -> 실패

목요일 -> 다시 머지 시도. c# 관련 문제라고 판단. 뭐 지금 내 어떤 파일들에서 지금 c#을 건들고 있고... 뭐 그러니 전부 빌드하지말고 일부만 해보아라... 내 원래 컴퓨터에 있는 지금 어떤 파일을 건들고 있으니 아예 그 파일명 변경으로 접근을 차단해라... 누겟 8.0이 없어 4.5 뭐 이런걸로 지금 하고 있어서 안되는거니 누겟8.0을 깔아라... 전부 시도. 비주얼 스튜디오도 삭제 후 다시 처음부터 설치 ... -> 정말 아무 소용 없었다. 정말 인생에서 제일 길게 집중했다. 근데 아무것도 안되고 원인도 모르고,, 여러 브랜치들을 이동하며 머지 시도. 내 브랜치와 머지하는 것만 안됨. 누겟8.0이 계속 날 괴롭힘. 이날도 거의 울면서 5시간 넘게 했다. 그치만 아무것도 되지않았다. 내 지식이 늘지도,, 문제가 해결되지도... 3일동안 머지로 인한 문제 때문에 미치는 줄 알았다. 결국 그냥 내 원래 파일 복사해놓고 메인기반 브랜치 새로 파서 하나하나 다 복붙했다. 하하

그리고 다 해결된 줄 알았는데, 원래 변위 맵을 통해 준 바닥 울퉁불퉁 효과는 적용되지 않고.. 새로운 텍스처로 해봐도 되지 않고 있고... sln에서 코드를 변경한 후 다시 빌드를 하면 이제 또 오류가 생긴다. 뭐 수정을 못하겠다. 사람 미치기 딱 좋은 주였다.

다음 주 할 일
로그인창 ui제작, 아이템 주문 경우 아이템 ui 사진 변경, 맵 계속 하기^^

특이사항